

Modal Numerosity Engine (MN) — Canvas (rc1)

One-liner. Learn a *count distribution* for “How many P-things could there have been?” by pushing a computably-defined world semimeasure through an isomorphism-invariant count functional, then validating via structure (replication→convolution) and effectivity (lower-semicomputability).

1) Executive summary

Problem. We ask counterfactual/possibility questions of the form: *How many X could there have been?* (colonies, causal centers, copies, etc.). People disagree because (a) different aspects/languages re-classify the same world differently; (b) models silently violate structural constraints (e.g., independence); (c) some proposed distributions aren’t even computably attainable.

Engine. Specify an aspect P (what is counted) and an isomorphism-invariant, computable counter N_P . Enumerate a family of computable laws $\{L_i\}$ with a lower-semicomputable prior π . Form the mixture world semimeasure $\lambda(w) = \sum_i \pi(i) L_i(w)$ and *push it forward* through N_P to get a count semimeasure λ_N over $\mathbb{N} \cup \{\infty\}$ and tail $\tau(n) = \sum_{m \geq n} \lambda_N(m)$. Validate by (i) **effectivity** (λ_N must be lower-semicomputable) and (ii) **structure** (replication→convolution when independence/additivity holds).

Deliverables.

- A reproducible **spec** (below) and a **reference skeleton** (API + demo wiring).
 - **Diagnostics:** convolution residuals, tail-class fit (polylog vs polynomial), language-invariance audits, and two **attainability certificates** (effectivity + structure).
-

2) Formal setup

2.1 Worlds, isomorphisms, aspects

- Worlds: finite descriptions $w \in W$.
- Isomorphism: an equivalence $\sim_{iso}$ (renamings/relabelings) under which counting is invariant.
- Aspect & predicate: choose a haecceity-free language $\mathcal{L}$ and definable predicate P .
- **Count functional:** $N_P : W \rightarrow \mathbb{N} \cup \{\infty\}$ with:
 - *Isomorphism invariance:* $w_1 \sim_{iso} w_2 \Rightarrow N_P(w_1) = N_P(w_2)$.
 - *Computability:* total computable (or partial with explicit timeout → see Extensions).

2.2 Laws and mixture

- Law family: enumerable $\{L_i\}$; each L_i a **lower-semicomputable semimeasure** over W (mass ≤ 1).
- Prior $\pi(i)$ is lower-semicomputable with $\sum_i \pi(i) \leq 1$.
- Mixture world semimeasure: $\lambda(w) = \sum_i \pi(i) L_i(w)$ (still lower-semicomputable).

- **Pushforward (image) measure** through N : $\lambda_N(n) = \lambda(\{w : N(w) = n\})$; tail $\tau(n) = \sum_{m \geq n} \lambda_N(m)$.

2.3 Structural operators

- Disjoint-union composition $\sqcup$ induces **replication**: $L^{(k)} = k$ independent copies.
- If **additivity** holds, $N(w_1 \sqcup w_2) = N(w_1) + N(w_2)$, then $p_{L^{(k)}} = p_L^{*k}$ (k-fold convolution of the base count distribution). Mixtures must lie in the **convex hull** of a convolution-closed family.

3) Guarantees (what you can certify)

3.1 Effectivity (Attainability-1)

If N is computable and λ is lower-semicomputable (l.s.c.), then the pushforward λ_N is **l.s.c.** (there exists a monotone lower approximation scheme). **Certificate**: any proposed target profile $q(n)$ that is *not* l.s.c. is unattainable under the MN kernel.

3.2 Structure (Attainability-2)

Under disjoint-union + independence + additivity, **replication** $\rightarrow$ **convolution**: for k copies, the observed count law must equal the k-fold convolution of the base law. Observed replication profiles outside the *convolution hull* are structurally unattainable.

4) Engine (buildable API)

4.1 `spec.language()` / `spec.aspect()`

Inputs: `language_id`, `aspect_id`, `P_def`, `iso_action` / `rename`.

Outputs: computable `N_P(w)` and `invariance_test(w, perm)`.

4.2 `spec.laws()`

Inputs: enumeration `i` $\mapsto$ `L_i`, l.s.c. prior `$\pi(i)$` , replication operators (`compose`, `independent_copy(k)`).

Outputs: streaming access to `$\lambda(w)$` via lower bounds.

4.3 `forward(max_steps | epsilon)`

Goal: accumulate monotone lower bounds for `$\lambda_N(n)$` and `$\tau(n)$` .

Outputs: `lambda_N_lower[n]`, `tail_lower[n]`, `assigned_mass_lower`, `unassigned_mass_upper`.

4.4 `fit(data)`

Likelihood over counts (hard, interval/graded, pairwise) $\rightarrow$ Bayesian update over `$\pi(i)$` and hyperparameters (discretized/particle approximations keep it computable).

4.5 `diagnose()`

- **Convolution check:** compare observed `$p(k)$` vs `$p\{k\}$` (L1/KL; report where residual concentrates).
- **Tail-shape check:** fit $\tau(n)$ to candidate classes (polylog vs polynomial).
- **Language invariance:** random renamings should leave `$N_P$` unchanged.

4.6 `attainability(q)`

- **Effectivity:** attempt to refute l.s.c. (show no computable lower approximation exists).
- **Structure:** test membership in the convolution hull via replication constraints/moments.

5) Tail-signature predictions (pre-registerable)

- **Polylog tails:** $\tau(n) \approx C/(\log n)^\beta$ under code-like priors.
- **Polynomial tails:** $\tau(n) \approx Cn^{-\gamma}$ under power priors.
- **Compression spikes:** excess mass near round numbers (10, 100, 1000, ...) from simplicity-biased mixtures.

6) Worked toy example (what the demo prints)

Using a “colonies” aspect (count is the integer at world key `colonies`) and two toy laws with lower bounds, the reference skeleton’s `demo` produces a pushforward with:

```
assigned_mass_lower ≈ 0.18
λ_N_lower ≈ {0:0.012, 1:0.024, 2:0.033, 3:0.042, 4:0.051, 5:0.018}
tail_lower ≈ {0:0.18, 1:0.168, 2:0.144, 3:0.111, 4:0.069, 5:0.018}
unassigned_mass_upper ≈ 0.82
```

This illustrates monotone lower-bound accumulation, tail computation, and explicit accounting of unassigned mass.

Files:

- Spec (Markdown): `mn_engine_spec.md`
 - Reference skeleton (Python): `mn_engine_skeleton.py`
-

7) Validation plan (fastest path)

1. **Synthetic sandbox:** implement simple generators L_i with known capacity bounds; verify tail-class predictions and replication→convolution.
 2. **Human study:** controlled scenarios (habitat/colonies). Manipulate replication (k copies) and aspect framing; test tail class, convolution residuals, and invariance pass rate. Pre-register exclusion rules and analysis.
-

8) Extensions (safe upgrades)

- **Partial computable counters:** add an explicit *unknown/timeout* bin; guarantees become “l.s.c. on known mass.”
 - **Handling ∞ :** track infinite counts separately; include in the tail.
 - **Policy packs:** run multiple (aspect, iso, prior) policies side-by-side and auto-compare tail class, invariance failures, and convolution residuals.
-

9) External references (for the mathematical underpinnings)

(Accessible sources linked; use textbooks for rigor where noted.)

- **Pushforward (image) measure.** Wikipedia overview; see also Bogachev's *Measure Theory* for a rigorous treatment.
https://en.wikipedia.org/wiki/Pushforward_measure
Bogachev, *Measure Theory*, Springer.
- **Convolution of independent integer-valued variables & PGFs.** Classic treatment in Feller; PGF property (product under independence).
Feller, *An Introduction to Probability Theory and Its Applications*, Vols I–II (Prentice Hall).
Probability-generating functions primer: https://grokipedia.com/page/Probability-generating_function
- **Algorithmic probability & lower-semicomputable semimeasures.**
Solomonoff (1964/1997 overviews):
 - “The Discovery of Algorithmic Probability” (1997): <https://raysolomonoff.com/publications/barc97.pdf>
 - “Does Algorithmic Probability Solve the Problem of Induction?” (1997): <https://raysolomonoff.com/publications/isis96.pdf>

Hutter, *Universal Artificial Intelligence* (Springer, 2005): <https://hutter1.net/ai/suaibook.pdf>
Scholarpedia summary noting universal (dominant) l.s.c. semimeasure: http://www.scholarpedia.org/article/Algorithmic_probability
Expository note on Levin's universal semimeasure: <https://arxiv.org/pdf/1508.05733v2>

- **Kolmogorov complexity background (for simplicity-biased priors).**

Li & Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed., Springer.
(Springer page) <https://link.springer.com/book/10.1007/978-3-030-11298-1>

- **Mixtures (identifiability background).**

Teicher, "Identifiability of Finite Mixtures," *Ann. Math. Statist.* 34(4):1265–1269 (1963): <https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-34/issue-4/Identifiability-of-Finite-Mixtures/10.1214/aoms/1177703862.full>

(These sources justify: (i) pushforward construction; (ii) convolution under independence; (iii) the use of lower-semicomputable semimeasures and universal dominance; (iv) simplicity-biased priors and tail-shape expectations.)

10) Next steps (pick one to ship)

- Wire a real **Aspect** (e.g., colonies with capacity constraints) and two concrete **Law** generators; expose a YAML/JSON config and dump `lambda_N_lower`, `tail_lower`, and diagnostics as a report.
- Add `diagnose_convolution()` to the demo to print L1 residuals for $k=2,3$.
- Package a prereg + IRB template using the Tail-signature checklist.

Citizen Gardens © 2025 CC-NC-ND 4.0